

Web Server Attacks - 1

Identifying Web Servers

The Server Response Header

The most direct fingerprinting signal is the `Server` header in an HTTP response. When you make any request to a web server, the server includes this header in its reply. Different server software formats it differently, and those formats are consistent enough to use as positive identifiers.

Run `curl` with the `-I` flag to request only the headers, not the response body:

```
root@ip-10-130-101-81:~# curl -sI http://10.130.132.250:80
HTTP/1.1 200 OK
Date: Mon, 22 Jun 2026 17:20:23 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Fri, 03 Apr 2026 18:12:44 GMT
ETag: "29af-64e9243796aa2"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

The `Server` header tells you the software and version outright

A hardened deployment might show only `Apache` or suppress the header entirely

Port	Server	Default Server Header
80	Apache2	Apache/2.4.x (Ubuntu)
8000	Python HTTP Server	SimpleHTTP/0.6 Python/3.xx.x
3000	Node.js Express	None (set by application)
8080	Nginx	nginx/1.xx.x

On a real Express application, the absence of a `Server` header is itself a signal. The reliable identifier for Express is the `X-Powered-By` header, which Express sets automatically unless the developer removes it.

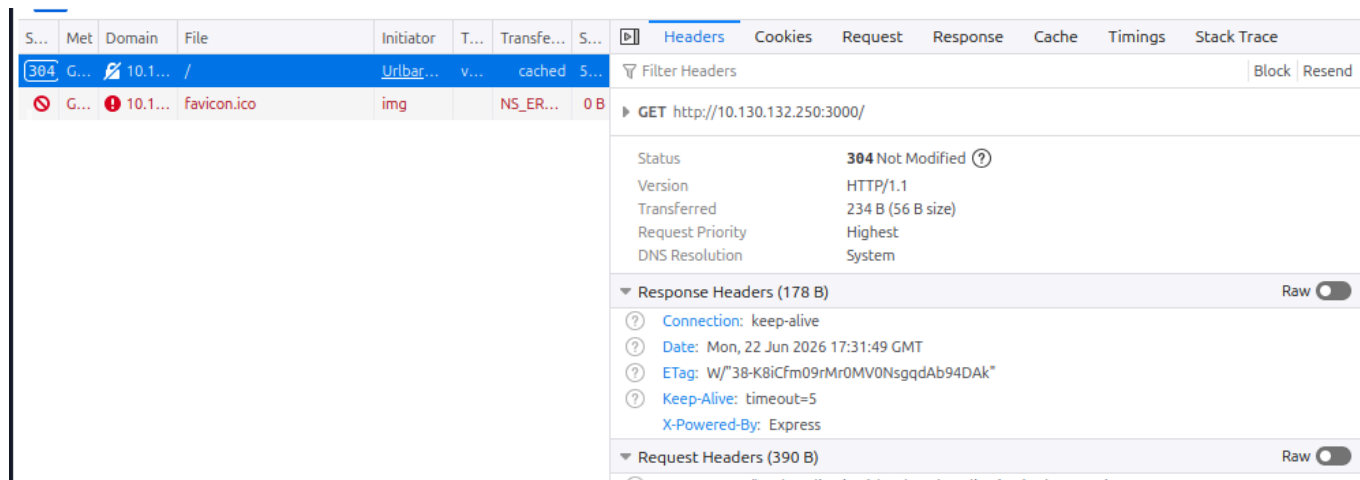
The X-Powered-By Header

Some frameworks add an `X-Powered-By` header that reveals the application layer behind the server. Express sets this by default

Unlike Apache or Nginx, which announce themselves in the `Server` header, Express relies on `X-Powered-By` as its identifier. Check for it on any port where `Server` is missing or generic.

Browser DevTools

working in a browser, the **Network** tab in DevTools provides the same header information without any additional tools. Open `http://10.130.132.250:3000` in Mozilla Firefox, then right-click anywhere on the page and choose **Inspect** (or press F12) to launch Developer Tools. Navigate to the **Network** tab and refresh the page to capture the requests. Select the main request from the list, and under the Headers section, view the Response Headers to inspect the server's response details.



Default Error Pages

The `-sI` flag sends a `HEAD` request, which returns headers only and no body. To see default error pages, you need a `GET` request switch to `-s` without the `-I`:

```
# HEAD request: headers only, no body
curl -sI http://10.130.132.250:PORT/

# GET request: full response including body
curl -s http://10.130.132.250:PORT/nonexistent-page-xyz
```

When you request a path that does not exist, most servers return a default 404 error page. Each server's default page has a distinctive look. Python is a plain text response. Nginx includes its version in the HTML footer. Apache includes its name in the page body. These differences let you fingerprint a server even when the `Server` header has been suppressed.

Check this across all four ports, and you will see how differently each server handles the same situation.

```
root@ip-10-130-101-81:~# curl -sI http://10.130.132.250:8000
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.12.3
Date: Mon, 22 Jun 2026 17:33:26 GMT
Content-type: text/html; charset=utf-8
Content-Length: 353

root@ip-10-130-101-81:~# curl -sI http://10.130.132.250:3000
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 56
ETag: W/"38-K8iCfm09rMr0MV0NsgqdAb94DAk"
Date: Mon, 22 Jun 2026 17:33:51 GMT
Connection: keep-alive
Keep-Alive: timeout=5

root@ip-10-130-101-81:~# curl -sI http://10.130.132.250:8080
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Mon, 22 Jun 2026 17:34:07 GMT
Content-Type: text/html
Content-Length: 136
Last-Modified: Fri, 03 Apr 2026 18:23:00 GMT
Connection: keep-alive
ETag: "69d00584-88"
Accept-Ranges: bytes
```

Python HTTP Server

Python ships with a built-in HTTP server that any developer can start with a single command. That convenience is exactly why it shows up on pentests.

```
# This command serves the current working directory over HTTP on port 8000
python3 -m http.server 8000
```

Developers use it to quickly share files, test static websites, or transfer something between two machines on the same network. The problem is that "quickly share files" sometimes becomes "accidentally exposed to the internet for six months", running on public-facing servers, internal network shares, and cloud instances where someone opened port 8000 in the firewall and forgot about it. The server has no access control, no authentication, and no logging beyond what the OS captures.

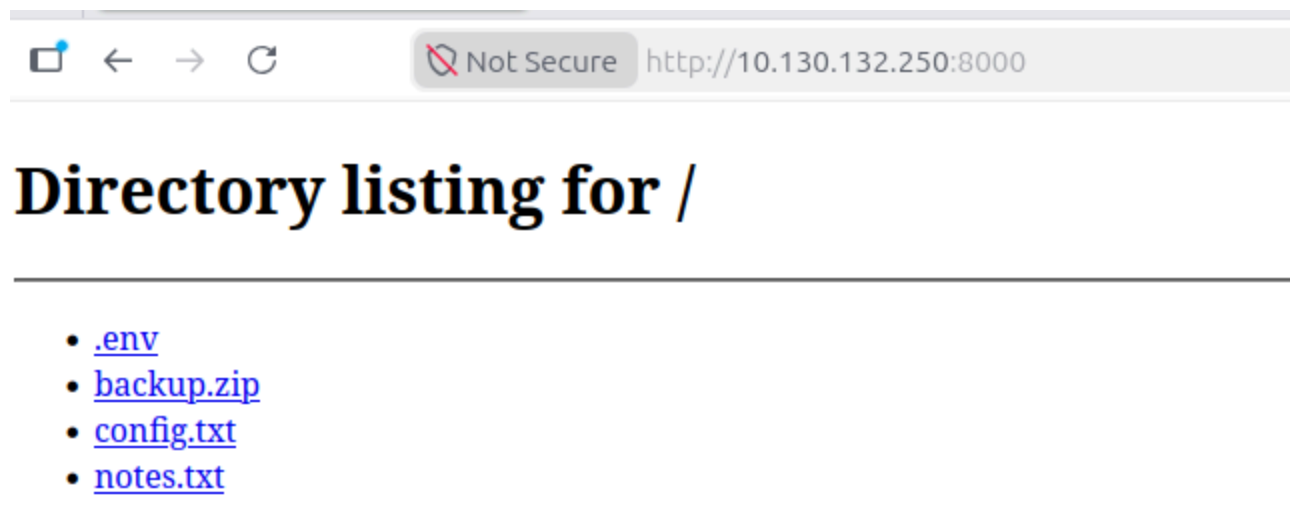
What It Serves

Python's HTTP server serves the entire working directory, including every file in it, including dotfiles like `.env`

If the file exists in the directory, anyone who can reach port 8000 can download it.

Directory Listing

When no `index.html` file is present in the directory, the Python HTTP server generates an HTML page listing every file it can see. Browse to the root of the server using the following command or directly browse `http://10.130.132.250:8000` in the browser:



Accessing Dotfiles

Dotfiles like `.env` are hidden from normal directory navigation on Linux, but Python's HTTP server does not respect that convention. It serves them like any other file. The `.env` file is a common target because developers use it to store environment-specific configuration, and that configuration often includes credentials

`curl -s http://10.130.132.250:8000/.env`

```
root@ip-10-130-101-81:~# curl -s http://10.130.132.250:8000/.env
SECRET_KEY=dev-secret-key-do-not-use
DATABASE_URL=postgres://webapp:S3cur3DBPass!@localhost/production
DEBUG=True
```

Downloading and Inspecting Archives

If you see a `.zip`, `.tar.gz`, or other archive in the directory listing, download it and inspect the contents

Developers sometimes leave backup archives in the same directory they are serving, which may contain source code, database dumps, or configuration files.

```
root@ip-10-130-101-81:~# curl -s http://10.130.132.250:8000/backup.zip -o backup.zip
root@ip-10-130-101-81:~# unzip backup.zip -d backup-contents/
Archive:  backup.zip
  inflating: backup-contents/db_dump.sql
root@ip-10-130-101-81:~# cat backup-contents/db_dump.sql
    id INTEGER PRIMARY KEY,
    username VARCHAR(50),
    email VARCHAR(100),
    created_at TIMESTAMP
);

INSERT INTO users VALUES (1, 'admin', 'admin@company.com', '2024-01-01 09:00:00');
INSERT INTO users VALUES (2, 'jsmith', 'jsmith@company.com', '2024-03-15 14:30:00');
INSERT INTO users VALUES (3, 'jdoe', 'jdoe@company.com', '2024-06-22 11:00:00');

CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2)
);

INSERT INTO products VALUES (1, 'Widget A', 9.99);
INSERT INTO products VALUES (2, 'Widget B', 19.99);

-- End of dump
-- flag: THM{py_server_exposed}
```

apache2

Apache is the most widely deployed web server in the world, which means it appears in nearly every engagement that touches web infrastructure. Its default Ubuntu configuration leaves several things enabled that testers regularly find and report. The three most common are directory listing on specific paths, the `server-status` module, and backup files sitting in the document root.

Version Disclosure

Start with the basics: check the `Server` header.

curl -SI <http://10.130.132.250:80> | grep -i server

```

root@ip-10-130-101-81:~# curl -SI http://10.130.132.250:80 | grep -i server
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Dload  Upload  Total   Dload  Upload    Spent    Left    Speed
  0 10671    0     0     0     0      0     0  --:--:--  --:--:--  --:--:--    0
Server: Apache/2.4.58 (Ubuntu)

```

Directory Listing

Apache's `Options +Indexes` directive tells the server to display a file listing when a directory does not have an `index.html` or similar default file. This is sometimes enabled intentionally for internal file share directories and accidentally left on paths that contain sensitive data.



Index of /files

Name	Last modified	Size	Description
 Parent Directory		-	
 employees.csv	2026-04-03 18:17	105	
 internal-notes.txt	2026-04-03 18:17	143	

Apache/2.4.58 (Ubuntu) Server at 10.130.132.250 Port 80

The mod_status Page

Apache includes a built-in status page powered by the `mod_status` module. When correctly configured, it is accessible only from localhost. When misconfigured with `Require all granted`, it is accessible from any IP.

Apache Server Status for 10.130.132.250 (via 10.130.132.250)

Server Version: Apache/2.4.58 (Ubuntu)
Server MPM: event
Server Built: 2026-03-05T17:31:54

Current Time: Monday, 22-Jun-2026 18:13:05 UTC
Restart Time: Monday, 22-Jun-2026 17:18:25 UTC
Parent Server Config. Generation: 1
Parent Server MPM Generation: 0
Server uptime: 54 minutes 40 seconds
Server load: 0.03 0.01 0.00
Total accesses: 10 - Total Traffic: 6 kB - Total Duration: 10
CPU Usage: u.01 s.06 cu0 cs0 - .00213% CPU load
.00305 requests/sec - 1 B/second - 614 B/request - 1 ms/request
1 requests currently being processed, 0 workers gracefully restarting, 49 idle workers

Slot	PID	Stopping	Connections		Threads		Async connections			
			total	accepting	busy	graceful	idle	writing	keep-alive	closing
0	787	no	0	yes	1	0	24	0	0	0
1	788	no	0	yes	0	0	25	0	0	0
Sum	2	0	0		1	0	49	0	0	0

.....
.....
.....

Scoreboard Key:
"_" Waiting for Connection, "s" Starting up, "r" Reading Request,
"w" Sending Reply, "k" Keepalive (read), "D" DNS Lookup,
"c" Closing connection, "L" Logging, "G" Gracefully finishing,
"I" Idle cleanup of worker, "." Open slot with no current process

Srv	PID	Acc	M	CPU	SS	Req	Dur	Conn	Child	Slot	Client	Protocol	VHost	Req
-----	-----	-----	---	-----	----	-----	-----	------	-------	------	--------	----------	-------	-----

The status page shows active connections and what paths they are requesting, the total number of requests served since the server started, worker states (idle, writing, reading, closing), and the server version and start time. On a production server, this leaks real-time information about what other users are doing and what internal paths exist. It also confirms the exact server version in the page header.

Info: `mod_status` is enabled by default on Ubuntu's Apache package and ships with a `Require local` restriction in `conf-available/security.conf`. However, a `Require all granted` directive anywhere in a virtual host configuration silently overrides that restriction, exposing `/server-status` to all IPs without touching the module config. Always check `/server-status` even on servers that appear to be using default settings.

Finding Unlinked Files with Gobuster

Not everything interesting is linked from a page or visible in a directory listing. Backup files, old configuration copies, and forgotten test files often sit in the document root with no links pointing to them. Tools like `Gobuster` discover these by guessing paths against a wordlist.


```
gobuster dir -u http://TARGET_IP:80 -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt -x bak,txt,html -t 20
```

```
# -u is the target URL
# -w is the wordlist path
# -x tells gobuster to also append these extensions to each word

gobuster dir -u http://TARGET_IP:80 -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt
-x bak,txt,html -t 20
```

Note: SecLists should be available at `/usr/share/wordlists/SecLists/` on the AttackBox. If you are using your own Kali VM and SecLists is not installed, run `sudo apt install seclists` to get it.

Tip: The full SecLists `common.txt` wordlist is large. With the `-x` flag adding three extension variants per word, the scan makes a significant number of requests and can take several minutes on a remote target. If you want a faster result, try running without `-x` first to find directories, then add `-x bak` for a targeted extension sweep.

=====

Gobuster v3.6

by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

=====

```
[+] Url:          http://10.130.132.250
[+] Method:       GET
[+] Threads:      20
[+] Wordlist:      /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.6
[+] Extensions:   bak,txt
[+] Timeout:      10s
```

=====

Starting gobuster in directory enumeration mode

=====

```
/.htpasswd      (Status: 403) [Size: 275]
/.htaccess      (Status: 403) [Size: 275]
/.htpasswd.bak  (Status: 403) [Size: 275]
/.htaccess.txt  (Status: 403) [Size: 275]
/.hta.txt       (Status: 403) [Size: 275]
/.htaccess.bak  (Status: 403) [Size: 275]
/.hta           (Status: 403) [Size: 275]
/.hta.bak       (Status: 403) [Size: 275]
/.htpasswd.txt  (Status: 403) [Size: 275]
/backup.bak     (Status: 200) [Size: 178]
/files          (Status: 301) [Size: 308] [--> http://10.130.132.250/files/]
/index.html     (Status: 200) [Size: 10671]
/javascript     (Status: 301) [Size: 313] [--> http://10.130.132.250/javascript/]
/server-status  (Status: 200) [Size: 17539]

Progress: 13965 / 13968 (99.98%)
```


When Gobuster finds a file with a `.bak` extension, that is almost always worth retrieving. Backup files frequently contain configuration snippets, credentials, or copies of source code.

Also, pay attention to `.htpasswd` files if they appear in the output. Apache uses `.htpasswd` to store usernames and hashed passwords for HTTP Basic Authentication. Finding an accessible `.htpasswd` file gives you a credential hash that can be cracked offline, and confirms that some part of the site uses Basic Auth, which tells you what paths are worth trying authenticated access on.

```
root@ip-10-130-101-81:~# curl -s http://10.130.132.250:80/backup.bak
# Apache config backup - DO NOT COMMIT
ServerName company.internal
DocumentRoot /var/www/html
# DB credentials below
# user: dbadmin pass: Backup2024!
# Last updated: 2024-11-15
```

Putting It Together

The pattern here is consistent across Apache investigations: check the version header, browse any directory that allows listing, visit `/server-status`, and use gobuster to find unlinked files. These four steps cover the majority of what a misconfigured Apache server will expose.

Node.js (Express)

Attackers look at **Node.js Express** applications specifically because developers often leave development-mode features enabled when they push to production. Debug endpoints, verbose error responses, and exposed environment variables all stem from the same habit: code that worked in development went live as-is. The result is an application that tells you exactly how it is built and, sometimes, what credentials it uses.

Framework Fingerprinting

curl -sI <http://10.129.128.26:3000>

```
root@ip-10-129-114-157:~# curl -sI http://10.129.128.26:3000
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 56
ETag: W/"38-K8iCfm09rMr0MV0NsgqdAb94DAk"
Date: Tue, 23 Jun 2026 17:19:49 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

X-Powered-By: Express in the response. Express sets this header automatically unless the developer explicitly disables it. This confirms you are dealing with an Express application, which tells you what to look for next.

Reading the Application Version

curl -s <http://10.129.128.26:3000>

```
root@ip-10-129-114-157:~# curl -s http://10.129.128.26:3000
{"status":"ok","app":"company-portal","version":"1.2.0"}root
#
```

Triggering Verbose Errors

Express's built-in error handler suppresses stack traces when `NODE_ENV` is set to `production`. In development mode, it passes them through

`NODE_ENV=production` check is only meaningful if no custom error handler overrides it.

curl -s <http://10.129.128.26:3000/api/users> | python3 -m json.tool

```
root@ip-10-129-114-157:~# curl -s http://10.129.128.26:3000/api/users | python3
-m json.tool
{
  "error": "connect ECONNREFUSED 127.0.0.1:5432",
  "stack": "Error: connect ECONNREFUSED 127.0.0.1:5432\n    at /opt/nodeapp/a
p.js:16:15\n    at Layer.handle [as handle_request] (/opt/nodeapp/node_modules/
express/lib/router/layer.js:95:5)\n    at next (/opt/nodeapp/node_modules/expres
s/lib/router/route.js:149:13)\n    at Route.dispatch (/opt/nodeapp/node_modules/
express/lib/router/route.js:119:3)\n    at Layer.handle [as handle_request] (/op
t/nodeapp/node_modules/express/lib/router/layer.js:95:5)\n    at /opt/nodeapp/no
de_modules/express/lib/router/index.js:284:15\n    at Function.process_params (/
opt/nodeapp/node_modules/express/lib/router/index.js:346:12)\n    at next (/opt/
nodeapp/node_modules/express/lib/router/index.js:280:10)\n    at expressInit (/o
pt/nodeapp/node_modules/express/lib/middleware/init.js:40:5)\n    at Layer.handl
[as handle_request] (/opt/nodeapp/node_modules/express/lib/router/layer.js:95:
)",
  "query": "SELECT * FROM users"
}
```

A 500 Internal Server Error from an API endpoint is worth investigating further.

In development mode, the response body often contains the error message, stack trace, and context about what the application was trying to do when it failed.

The stack trace is the key finding here, as it reveals internal file paths, module names, and, sometimes, the database query that triggered the failure

Enumerating Routes via Debug Endpoints

One of the most useful things a misconfigured Express application can do is tell you all of its own routes. Developers sometimes build a debug endpoint that lists every registered route for convenience during development, then forget to remove it before going live.

curl -s <http://10.129.128.26:3000/api/routes>

```
}
root@ip-10-129-114-157:~# curl -s http://10.129.128.26:3000/api/routes
[{"method": "GET", "path": "/"}, {"method": "GET", "path": "/api/users"}, {"method": "GET", "path": "/api/routes"}, {"method": "GET", "path": "/api/debug/env"}]
```

If this endpoint exists, the response is a list of every path the application handles. This saves enumeration time and shows you which endpoints exist before you spend time guessing them with Gobuster.

Note: The `/api/routes` endpoint works by reading Express's internal `app._router.stack` property to enumerate registered routes. This is a recognised Express misconfiguration pattern, but `app._router.stack` is an internal property whose structure can vary between Express versions, and Express 5 changed the router internals enough to break implementations that relied on the Express 4 structure. If you encounter a route endpoint that returns an unexpected format or an error, the application may be running a different Express version than the one this lab uses.

Exposed Environment Variables

Environment variables in Node.js applications often contain database credentials, API keys, and configuration flags. A debug endpoint that returns `process.env` is a significant finding

curl -s <http://10.129.128.26:3000/api/debug/env>

```
root@ip-10-129-114-157:~# curl -s http://10.129.128.26:3000/api/debug/env
{"NODE_ENV": "development", "DB_PASSWORD": "NodeDBPass2024!", "PORT": "3000", "DB_HOST": "localhost:5432", "APP_NAME": "company-portal"}root@ip-10-129-114-157:~#
```

If the response includes `DB_PASSWORD`, `SECRET_KEY`, or similar fields, document them.

The `NODE_ENV` value is also telling. `NODE_ENV=development` on a production server is a signal that the application was deployed without proper hardening.

Static File Serving

Express applications commonly use the `express.static()` middleware

The static files route, if it exists, serves everything in a directory. Client-side JavaScript files sometimes contain API endpoint URLs, internal hostnames, or debug flags embedded as constants.

Once you know the routes from the `/api/routes` endpoint, check what is being served statically

curl -s <http://10.129.128.26:3000/static/config.js>

```
432", "APP_NAME": "company-portal"}root@ip-10-129-114-157:~# curl -s http://10.129.128.26:3000/s
curl -s http://10.129.128.26:3000/static/config.js
// Client-side configuration
const API_BASE = 'http://internal-api.company.local:8080';
const DEBUG = true;
const VERSION = '1.2.0';
// flag: THM{node_debug_exposed}
root@ip-10-129-114-157:~#
```

configuration files served as static assets are easy to overlook because they are technically meant to be public (the browser needs them). But "meant to be public" is different from "contains only public information."

Note: `express.static()` silently returns a 404 for dotfiles (files starting with `.`) by default. This is the opposite of Python's HTTP server, which serves dotfiles like any other file. If you are looking for a `.env` or similar file on an Express static route and get a 404, that does not mean the file is absent; it means the middleware is blocking it. You would need shell access or a different route to reach it.

Putting it All Together

Take a step back and look at what we just did. We started with header inspection to confirm the framework, then triggered an error to see what the application leaks in failure mode, then used a debug endpoint to enumerate routes, then checked for credential exposure in environment variables, and finally followed the route list to the static files where the flag lives. This pattern works because each step narrows the scope for the next one. Headers tell you the framework. Errors tell you the internals. Debug endpoints tell you the routes. Static files tell you what the developers assumed was safe to expose.

Nginx

The categories of misconfiguration version disclosure, directory listing, and exposed status pages appear again, but the specific directives and paths differ.

Nginx occupies a different space from Apache and Node.js. It is most commonly used as a reverse proxy, a load balancer, or a high-performance static file server. In production environments, Nginx often sits in front of an application server and handles the public-facing traffic.

A misconfigured Nginx instance can expose internal structure, reveal operational data, or make files accessible that should not be.

Version Disclosure

The pattern is the same as before: start with the `Server` header.

```
curl -sI http://10.129.128.26:8080 | grep -i server
```

```
root@ip-10-129-114-157:~# curl -sI http://10.129.128.26:8080 | grep -i server
Server: nginx/1.24.0 (Ubuntu)
```

Knowing the exact version is useful when checking the Nginx changelog for security fixes or when building an engagement report.

The `server_tokens` directive controls both the `Server` header and the version string in default error pages simultaneously; setting `server_tokens off` suppresses the version from both places at once. If the `Server` header is suppressed, requesting a non-existent path will confirm whether `server_tokens` is truly off or just partially configured?

```
curl -s http://10.129.128.26:8080/nonexistent-path
```

```
root@ip-10-129-114-157:~# curl -s http://10.129.128.26:8080/nonexistent-path
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
```

Directory Listing with Autoindex

Nginx does not enable directory listing by default. When a developer wants to expose a directory listing, they add `autoindex` to a location block in the configuration

```
location /files/ {
    autoindex on;
    root /var/www/nginx/;
}
```

This is a documented Nginx configuration option and appears in legitimate file-sharing setups. The misconfiguration is using it on a path that contains sensitive files, or leaving it enabled on a production server without access controls.

```
curl -s http://10.129.128.26:8080/files/
```

```

root@ip-10-129-114-157:~# curl -s http://10.129.128.26:8080/files/
<html>
<head><title>Index of /files/</title></head>
<body>
<h1>Index of /files/</h1><hr><pre><a href="..">../</a>
<a href="deploy-notes.txt">deploy-notes.txt</a>
18:23          148
<a href="old-backup.tar.gz">old-backup.tar.gz</a>
18:23          236
<a href="server-config.txt">server-config.txt</a>
18:23          135
</pre><hr></body>
</html>
03-Apr-2026
03-Apr-2026
03-Apr-2026

```

The nginx_status Endpoint

Nginx's `stub_status` module exposes real-time connection metrics at a configurable URL. The secure configuration restricts access to localhost only. The misconfigured version allows access from any IP

```

location /nginx_status {
    stub_status;
    allow all; # Should be: allow 127.0.0.1; deny all;
}

```

curl -s http://10.129.128.26:8080/nginx_status

```

root@ip-10-129-114-157:~# curl -s http://10.129.128.26:8080/nginx_status
Active connections: 1
server accepts handled requests
 4  4  4
Reading: 0 Writing: 1 Waiting: 0

```

The three numbers on the second line are in order: total accepted connections, total handled connections, and total requests since the server started.

The third line breaks down currently active connections by state

On a real engagement, an exposed `/nginx_status` endpoint is a finding because it confirms the internal monitoring setup and may indicate other monitoring endpoints are similarly exposed.

Putting It All Together

The Nginx investigation follows a parallel structure to the Apache task. Check the version header, browse any directories with autoindex enabled, and check for the status endpoint. The specific paths and configuration directives differ, but the investigative approach is the same.

✓ **Tip:** Nginx configuration files live in `/etc/nginx/` on Ubuntu. If you ever have shell access to a machine running Nginx, reading the site configuration in `/etc/nginx/sites-available/` will show you exactly what directories are exposed and what modules are enabled.

Common Misconfigurations Across Servers

Security Headers

Security headers are HTTP response headers that instruct the browser on how to handle the page content. They protect against a range of client-side attacks, including clickjacking, MIME sniffing, and cross-site scripting. None of the servers in this lab has been configured to send these headers, which is the default state for all four server types.

Header	What It Protects Against	Example Value
<code>X-Frame-Options</code>	Clickjacking (prevents the page from being embedded in an iframe on another domain)	<code>DENY</code> or <code>SAMEORIGIN</code>
<code>X-Content-Type-Options</code>	MIME sniffing (prevents the browser from guessing content types)	<code>nosniff</code>
<code>Content-Security-Policy</code>	Restricts where scripts, stylesheets, and other resources can load from	<code>default-src 'self'</code>
<code>Referrer-Policy</code>	Controls what is sent in the <code>Referer</code> header when navigating to another page	<code>no-referrer</code> or <code>strict-origin</code>
<code>Strict-Transport-Security</code>	Forces HTTPS for subsequent requests (only meaningful on HTTPS servers)	<code>max-age=31536000</code>

Note: `X-Frame-Options` is technically superseded by `Content-Security-Policy: frame-ancestors`, which provides finer-grained control. Hardened modern sites may handle clickjacking protection through CSP alone and intentionally omit `X-Frame-Options`. When writing findings, check for both.

Audit each server with `curl`

```
for port in 80 8000 3000 8080; do echo "=== Port $port ==="; curl -sI http://10.129.128.26:$port/ | grep -iE "x-frame-options|x-content-type|content-security-policy|strict-transport|referrer-policy" || echo "(no security headers found)"; done
```



```

root@ip-10-129-114-157:~# for port in 80 8000 3000 8080; do echo "=== Port $port ==="; curl -I http://10.129.128.26:$port/ | grep -iE "x-frame-options|x-content-type|content-security-policy|strict-transport|referrer-policy" || echo "(no security headers found)"; done
=== Port 80 ===
(no security headers found)
=== Port 8000 ===
(no security headers found)
=== Port 3000 ===
(no security headers found)
=== Port 8080 ===
(no security headers found)

```

When `grep` returns no output, it means none of those headers is present. Do this across all four ports, and you will see that the default configurations for all four server types share this gap. Security headers require active configuration. They are never present by default.

Note: `Strict-Transport-Security` only applies over HTTPS connections. Since this lab uses HTTP, its absence is expected and is noted here for completeness rather than as an active misconfiguration.

Automated Scanning with Nikto

Nikto is a web server scanner that checks for known misconfigurations, outdated software, exposed admin interfaces, and missing security headers. It is not subtle; it generates a lot of traffic and is easy to detect. That makes it appropriate for authorised testing, not stealth. Run it against the Apache server

nikto -h <http://10.129.128.26:80> -nointeractive

```

root@ip-10-81-64-63:~# nikto -h http://10.129.128.26:80 -nointeractive
- Nikto v2.1.5
-----
+ Target IP:          10.129.128.26
+ Target Hostname:    10.129.128.26
+ Target Port:        80
+ Start Time:         2026-04-11 09:16:09 (GMT1)
-----
+ Server: Apache/2.4.58 (Ubuntu)
+ Server leaks inodes via ETags, header found with file /, fields: 0x29af 0x64e9243796aa2
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: HEAD, GET, POST, OPTIONS
+ OSVDB-561: /server-status: This reveals Apache information. Comment out appropriate line in
+ OSVDB-3268: /files/: Directory indexing found.
+ OSVDB-3092: /files/: This might be interesting...
+ 6544 items checked: 0 error(s) and 6 item(s) reported on remote host
+ End Time:          2026-04-11 09:16:18 (GMT1) (9 seconds)
-----
+ 1 host(s) tested

```

The `-nointeractive` flag suppresses prompts so the scan runs without waiting for input. Nikto will check hundreds of common paths and patterns. Look for findings that start with `+` in the output.

On a misconfigured Apache server like this one, expect Nikto to flag the exposed `/server-status` page, the `backup.bak` file in the document root, the directory listing on `/files/`, and multiple missing security header warnings.

Tip: Nikto's output can be verbose. The `-Tuning` option lets you restrict which checks it runs. For a quick scan, `nikto -h TARGET -Tuning 123` covers the most common findings without running the full signature set. Note that tuning codes are concatenated, not comma-separated.

Patterns That Apply Everywhere

Misconfiguration	Apache	Python HTTP	Node.js	Nginx
Version disclosure in headers	Yes	Yes	Partial	Yes
Directory listing	<code>/files/</code>	Root path	N/A	<code>/files/</code>
Exposed status or debug endpoint	<code>/server-status</code>	N/A	<code>/api/debug/env</code> , <code>/api/routes</code>	<code>/nginx_status</code>
Sensitive files accessible	<code>backup.bak</code> , <code>internal-notes.txt</code>	<code>.env</code> , <code>notes.txt</code> , <code>backup.zip</code>	<code>config.js</code>	<code>server-config.txt</code> , <code>deploy-notes.txt</code>
Missing security headers	All	All	All	All

nikto -h <http://10.129.174.162:80> -nointeractive

```

root@ip-10-129-116-183:~# nikto -h http://10.129.174.162:80 -nointeractive
+ Nikto v2.1.5
+-----+
+ Target IP:      10.129.174.162
+ Target Hostname: ip-10-129-174-162.eu-west-3.compute.internal
+ Target Port:    80
+ Start Time:     2026-06-23 18:29:58 (GMT0)
+-----+
+ Server: Apache/2.4.58 (Ubuntu)
+ Server leaks inodes via ETags, header found with file /, fields: 0x29af 0x64e9243796aa2
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: GET, POST, OPTIONS, HEAD
+ OSVDB-561: /server-status: This reveals Apache information. Comment out appropriate line in ht
pd.conf or restrict access to allowed hosts.
+ OSVDB-3268: /files/: Directory indexing found.
+ OSVDB-3092: /files/: This might be interesting...
+ 6544 items checked: 0 error(s) and 6 item(s) reported on remote host
+ End Time:      2026-06-23 18:30:06 (GMT0) (8 seconds)
+-----+
+ 1 host(s) tested

```

Key Takeaways

- Response headers are the fastest fingerprinting surface. The `Server` and `X-Powered-By` headers reveal software type and sometimes version before you have made a single request to an actual application path.
- Python's built-in HTTP server has no access controls of any kind. When it appears in an engagement, assume the entire working directory is readable, including dotfiles such as `.env`.
- Apache's `mod_status` and directory listing are enabled by default on Ubuntu installs and require active configuration to restrict them. Their presence is common and worth checking on every Apache engagement.
- Node.js Express applications in development mode leak stack traces, route lists, and environment variables. Each of those is a finding in its own right, and together they give a complete picture of the application's internals.
- Nginx `autoindex` and `stub_status` mirror Apache's directory listing and `mod_status` patterns. The directives differ, but the investigation approach is the same.
- Security headers are absent by default on all four server types. An audit with `curl -sI` and a `grep` for the header names is a fast, reliable check that belongs in every web server engagement.